

Requirement Writing Tips



b7b0630a-816a-48a3-91c5-d9db6fc5...

PDF

explain everything here in details

Absolutely! Here's a **detailed explanation of everything in your document**, breaking down every example and guideline into clear, exam-ready notes with reasoning, best practices, and practical tips for writing better requirements.

1. Example 1 – Ambiguous Functional Requirement

Bad Example

- *"The system shall provide fast search functionality with good results."*

Why is this Bad?

- **Ambiguity:** "Fast" isn't defined—is it 1 second or 10 seconds?
- **Unmeasurable:** Can't objectively test what's "fast enough".
- **Vague Criteria:** "Good results" is subjective; what makes results "good"?
- **Incomplete:** No info on what content is searchable, how ranking works.
- **Unverifiable:** No way to test or confirm if requirement is met.

Better Version

Users will:

- **Response time:** Receive search results within 3 seconds after entering up to 5 keywords.
- **Result display:** See the top 20 matches in a ranked list.
- **Ranking algorithm:** Results are prioritized by matches in product titles (primary), then descriptions (secondary).
- **Performance at scale:** Consistent performance even when searching 100,000 products.

Key Learning:

→ Functional requirements must be specific, measurable, and testable.

2. Example 2 – Vague Non-functional Requirement

Bad Example

- *"The system must have good uptime."*

Why is this Bad?

- **Subjective:** What does "good" mean? Is it 90% or 99.9%?
- **No measurement period:** Is uptime measured monthly, yearly?
- **No exclusions:** Does scheduled maintenance count against uptime?
- **No recovery metrics:** How fast should the system recover after failure?
- **Not testable:** No way to verify satisfaction.

Better Version

Users will:

- **Availability guarantee:** Access app 99.5% of the time each month.
- **Maintenance policy:** Scheduled maintenance max 4 hours/month, announced 5 days early, not counted against availability.
- **Recovery time:** All core functions restored within 30 minutes after any outage.

Key Learning:

→ Non-functional requirements need clear metrics, scope, and boundaries.

3. Example 3 – Unmeasurable Usability Requirement

Bad Example

- *"The user interface must be user-friendly."*

Why is this Bad?

- **Subjective:** "User-friendly" means different things to different people.
- **No target users:** Who are we making it friendly for?
- **No metrics:** No way to measure "friendliness".
- **No specifics:** Which usability aspects matter?
- **Unverifiable:** Can't confirm completion.

Better Version

First-time users with no prior training will:

- **Task completion efficiency:** Submit orders in under 5 minutes.
- **Error tolerance:** Make no more than 2 errors during order submission.
- **User satisfaction:** At least 85% of users rate the UI as 'easy' or 'very easy' (5-point scale).

Key Learning:

→ Usability requirements must be measurable and focused on outcomes for specific users.

4. Example 4 – Incomplete Security Requirement

Bad Example

- *"The system shall be secure."*

Why is this Bad?

- **Too general:** Which parts must be secure?
- **No standards:** What level of security or compliance?
- **No threat model:** What are we protecting against?
- **No authentication details:** How will users be verified?
- **No verification:** Can't test if security is "enough".

Better Version

Users will:

- **Secure communications:** Use HTTPS with TLS 1.3 encryption for all data.
- **Data protection:** Personal data stored with AES-256 encryption.
- **Authentication security:** Passwords ≥ 8 chars, include numbers/specials; lockout after 5 failed logins.
- **Payment security:** Payments via PCI-DSS compliant system.

Key Learning:

→ Security requirements need details: specific protocols, standards, and scenarios.

5. Example 5 – Technically Infeasible Requirement

Bad Example

- *"The system must be 100% error-free and never experience downtime."*

Why is this Bad?

- **Unrealistic:** No system can be 100% error-free or always up.
- **Unfeasible:** Hardware failures happen.
- **Economically impractical:** Cost to reach near-perfection is huge.
- **Ignores tradeoffs:** No mention of cost/reliability balance.
- **Impossible expectations:** Misleads stakeholders.

Better Version

Users will:

- **System availability:** Use the app reliably 8am–8pm with 99.9% availability.
- **Error rate:** Critical errors in fewer than 1 in 1,000 transactions.
- **Error handling:** Users notified of errors; issues resolved within 1 business hour; proactive monitoring for issues.

Key Learning:

→ Requirements must be feasible, realistic, and balanced with business priorities.

6. Example 6 – Solution Constraint Instead of Requirement

Bad Example

- *“The system shall use a MySQL database and PHP for the backend.”*

Why is this Bad?

- **Specifies implementation:** Tells how, not what is needed.
- **Limits innovation:** Prevents using better/new tech.
- **Not a requirement:** It's a tech choice, not a user need.
- **Obsolescence risk:** Tech changes faster than user needs.
- **Focuses on means, not ends:** Doesn't explain why this tech.

Better Version

Users will:

- **Data access performance:** Access individual customer records in <0.5s; generate reports (up to 10,000 records) in <3s.

Key Learning:

→ State *what* is needed, not *how* to achieve it—unless it's a justified constraint.

7. Example 7 – Multiple Requirements in One

Bad Example

- *“The system shall allow users to manage their profile, send messages, upload documents, and generate reports while maintaining privacy and security.”*

Why is this Bad?

- **Not atomic:** Actually 5+ requirements in one.
- **Hard to trace:** Can't track each feature separately.
- **All-or-nothing:** Can't prioritize/implement individually.
- **Hard to test:** Can't verify each item separately.
- **Ambiguous responsibility:** Which team does what?

Better Version

Split into unique requirements:

- **REQ-1.1:** Users can view/edit profile info in a profile section.
- **REQ-1.2:** Users can send private messages to other users.
- **REQ-1.3:** Users can upload documents (PDF/DOC/DOCX).
- **REQ-1.4:** Users can generate activity reports with custom date ranges.
- **REQ-1.5:** Users control privacy by setting visibility permissions.

Key Learning:

→ Requirements must be atomic, uniquely traceable, and individually testable.

Characteristics of Good Requirements

PART 1

1. **Clear & Unambiguous:** Only one interpretation. No vague words like "fast", "user-friendly", or "secure".
2. **Complete:** Contains all necessary info—inputs, outputs, and conditions.
3. **Verifiable & Testable:** Can be objectively checked; measurable criteria.
4. **Consistent:** No contradictions with other requirements.

PART 2

5. **Atomic:** Does only one thing; can be implemented and validated on its own.

- 6. **Traceable:** Linked to business needs above and code/tests below.
- 7. **Feasible:** Can be built with available tech, within time/resources.
- 8. **Necessary:** Solves an actual need; not redundant.

PART 3

- 9. **Implementation-Free:** Focuses on what, not how (unless justified).
- 10. **Quantifiable:** Uses metrics—time, %, counts—over qualitative terms.
- 11. **User-Focused:** Describes from the user’s perspective, not the system’s.
- 12. **Prioritized:** Indicates relative importance for planning.

Summary Table

Poor Requirement	Why It’s Bad	How To Fix It / Good Example
Fast search with good results	Ambiguous, vague, not testable	Define search speed, number of results, ranking, scale
Good uptime	Subjective, no metrics, not testable	Define % uptime, exclude maintenance, recovery time
User-friendly UI	Subjective, unmeasurable, generic	Define target user, task time, error rates, satisfaction
Secure system	General, lacks specifics, not testable	Define encryption, authentication, payment security
100% error-free	Unrealistic, not feasible	Define realistic error rates, downtime, error handling
Use MySQL & PHP	Solution constraint, not user need	State performance needs, not implementation
Manage profile + ...	Multiple requirements bundled	Split into atomic, testable, traceable requirements

Key Takeaways for Exams and Real Projects

- Always **avoid vague, subjective, or unmeasurable statements** in requirements.
 - **Specify measurable, testable, and realistic criteria** for every requirement.
 - Requirements should **focus on WHAT needs to be achieved**—not HOW to do it (unless it's a necessary constraint).
 - Make sure **every requirement is atomic** (can stand alone), **traceable**, and **prioritized**.
 - **Reference standards, metrics, and user scenarios** wherever possible.
-

If you need **sample exam answers, diagrams, or more practical scenarios** based on this, just let me know!

give me exam style questions and answers

Sure! Here are **exam-style questions** based on your PDF content about writing better requirements, each with **model answers**. These include MCQ, short answer, and scenario-based questions—perfect for BCS, university, or SWE exam prep.

1. Multiple Choice Questions (MCQ)

Q1.

Which of the following is the BEST example of a well-written requirement?

- a) The system shall be user-friendly.
- b) The system must have good uptime.
- c) The user shall receive search results within 3 seconds after entering up to 5 keywords.
- d) The backend shall use PHP and MySQL.

Answer:

c) The user shall receive search results within 3 seconds after entering up to 5 keywords.

Q2.

What is the main problem with the requirement: "The system must be 100% error-free and never experience downtime"?

- a) It is ambiguous.
- b) It is not feasible or realistic.
- c) It is incomplete.
- d) It is consistent.

Answer:

b) It is not feasible or realistic.

2. Short Answer Questions

Q3.

List four characteristics of a good requirement specification.

Answer:

- Clear and unambiguous
 - Complete
 - Verifiable and testable
 - Consistent
-

Q4.

Why is it a bad practice to specify a particular technology (e.g., "The system shall use MySQL database and PHP for the backend") in requirement specifications?

Answer:

It focuses on *how* to implement the system rather than *what* the system should achieve. This limits developer innovation, risks obsolescence, and may not reflect the actual needs of users

or the business. Technology choices should be made during the design phase, unless there is a justified constraint.

3. Scenario-Based Questions

Q5.

Given this requirement:

“The system shall allow users to manage their profile, send messages, upload documents, and generate reports while maintaining privacy and security.”

- a) Identify two issues with this requirement.
- b) Rewrite the requirement in a better way.

Answer:

- a)
 - The requirement is not atomic (bundles multiple requirements in one statement).
 - Difficult to trace and test each feature separately.

b)

Split into separate, atomic requirements, e.g.:

- Users shall be able to manage their profile via a dedicated profile page.
- Users shall be able to send private messages to other users.
- Users shall be able to upload documents in PDF, DOC, or DOCX formats.
- Users shall be able to generate activity reports with custom date ranges.
- Users shall set privacy permissions for their data.

4. Application/Reasoning Questions

Q6.

A stakeholder gives you the following requirement: “The UI should be intuitive and easy to use.” How would you clarify and improve this requirement?

Answer:

Replace subjective terms like “intuitive” with measurable criteria. For example:

- First-time users without training must complete the primary workflow (e.g., order submission) in under 5 minutes.
 - Users should make no more than 2 errors per process.
 - At least 85% of users must rate the UI as ‘easy’ or ‘very easy’ in post-task surveys.
-

Q7.

Explain the difference between “atomic” and “traceable” requirements, giving an example for each.

Answer:

- **Atomic:** Each requirement should address only one function or need, so it can be implemented, tested, and modified independently (e.g., “The user shall be able to reset their password.”).
 - **Traceable:** Each requirement should be linked to a specific business or user need and be trackable throughout the project lifecycle (e.g., a requirement traced back to a user story about account recovery).
-

5. True/False

Q8.

Requirements should always specify *how* the solution will be built.

(True/False)

Answer:

False. Requirements should focus on *what* the system must do, not *how* to do it (except when implementation constraints are necessary).

6. Fill in the Blanks

Q9.

A good requirement is _____, meaning it can be objectively validated through testing, and _____, meaning it does not conflict with other requirements.

Answer:

verifiable (or testable); consistent

7. Open-Ended (5 Marks)

Q10.

Describe three consequences of writing poor or ambiguous requirements in a software project.

Answer:

- 1. Miscommunication:** Developers may interpret requirements differently, leading to inconsistent implementation.
 - 2. Increased Costs:** Vague requirements can cause rework, delays, and additional costs due to misunderstandings or incomplete features.
 - 3. Difficult Testing:** Ambiguous requirements are hard to test, making it impossible to confirm if user needs are met, potentially resulting in project failure.
-

Let me know if you want **long-form questions, case studies, or customized practice sets!**